

# Automatic Speech Recognition with Transformers

Applied Deep Learning — Chapter 23

---

Yin Yang

Foundation Books Project

# Roadmap

Running Example and Goals

Audio as Data

What ASR Predicts and How It Aligns

Evaluation: WER, CER, RTF

End-to-End Objectives: CTC, Attention, Transducer

Transformer ASR Architectures

Audio-Language Models and Prompted ASR

Case Study: Qwen3-ASR

Pipeline, Errors, and Recipe

Summary

## Running Example and Goals

---

# What ASR Is, and Why a Transcript Is Evidence

- **ASR** maps a finite spoken audio signal to a text sequence.
- Running example: short multilingual lecture clips, transcribed for review, search, and downstream summarization.
- A transcript that “looks fine” is not a result.
  - Reference text + hypothesis text + speed on named clips = evidence.

A serious ASR claim names the clips, the reference, the hypothesis, the metric, and the runtime — not just an impression.

# The Manifest: Small, Visible, Reproducible

| Audio id     | Language       | Duration | Condition     | Reference excerpt                                      |
|--------------|----------------|----------|---------------|--------------------------------------------------------|
| lec-clean-01 | English        | 8.4 s    | clean lecture | Gradient descent updates the weights after each batch. |
| lec-noisy-02 | English        | 10.2 s   | room echo     | The validation set guides model selection.             |
| lec-ar-03    | Arabic/English | 7.8 s    | code-switch   | We compare WER and CER for the transcript.             |
| lec-term-04  | English        | 6.5 s    | domain term   | Qwen3-ASR is a transformer-based recognizer.           |

Inspect duration, sample rate, and reference text before opening a model. Domain-sensitive errors (descent → decent, batch → badge) hide inside “looks fine” transcripts.

# Offline vs Streaming ASR

**Offline** Receives a complete clip, then returns a transcript.

**Streaming** Receives audio in chunks and emits partial / final text while the speaker is still talking.

Speed means different things in each mode:

- Offline: real-time factor (RTF) over whole clips.
- Streaming: time to first partial output, finalization delay.

**Conservative recipe:** start from a pretrained transformer ASR baseline, build a small validation set, change one factor at a time.

# Audio as Data

---

# From Waveform to Frame Features

- Waveform: ordered sequence of amplitude samples.
- At 16,000 Hz, one second of audio = 16,000 samples; a 3-second clip = 48,000 samples.
- Most recognizers do not reason over isolated samples.

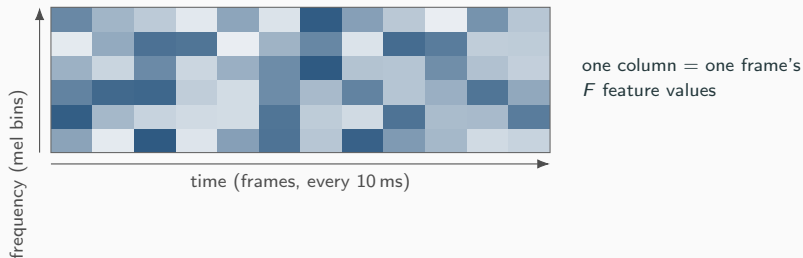
Frames: 25 ms windows every 10 ms. A 3.000 s clip yields

$$1 + \left\lfloor \frac{3.000 - 0.025}{0.010} \right\rfloor = 298 \text{ frame positions.}$$

Audio length grows fast. Seconds become tens of thousands of samples, hundreds of frame positions. Long sequences make attention expensive.



# Spectrogram: Frequency $\times$ Time Grid



Each cell stores energy at one frequency bin during one short window. This grid is the input to the encoder — not the raw waveform.

## Feature Tensor Shapes and Loading Choices

Batch of  $B = 4$  clips,  $T = 300$  frames,  $F = 80$  log-mel features per frame:

$$\text{shape } (B, T, F) = (4, 300, 80).$$

Loading is part of the run, not invisible cleanup:

- Sample rate (loaded vs expected); resampling rule.
- Channel mixing to mono; leading-silence trimming.
- Volume normalization or denoising.

A clipped or aggressively denoised signal can change the transcript. Hidden resampling rules destroy reproducibility before model debate begins.

## What ASR Predicts and How It Aligns

---

# Output Vocabulary and Tokens

ASR turns long audio into shorter text. Two coupled problems:

- **Recognize** which tokens were spoken.
- **Align** time positions to output tokens.

Output vocabulary may include characters, subwords, words, language tags, timestamp symbols, special tokens.

Example: “deep learning”

- Char level: d,e,e,p,space,l,e,a,r,n,i,n,g
- Subword level: deep, learning

Subwords are efficient but a rare technical token (e.g. Qwen3-ASR) can dominate CER even when the sentence sounds plausible.

# The Output Contract

Decide *before* evaluation:

- Text only, or text + language tag?
- Segment timestamps? Word-level timestamps?
- How are no-speech regions handled?

**Forced alignment** Given audio + known transcript, predict where words occur in time.

**Language ID** Predict spoken language; can run jointly or separately.

Silence that produces a fluent sentence nobody said is a hallucinated transcript — not a harmless formatting issue.

## Evaluation: WER, CER, RTF

---

## Word Error Rate by Hand

Edit operations: substitution  $S$ , deletion  $D$ , insertion  $I$ .

$$\text{WER} = \frac{S + D + I}{N}, \quad N = \# \text{ words in reference.}$$

Reference: “train the model today.” Hypothesis: “train model today.”

$$S = 0, D = 1, I = 0, N = 4 \Rightarrow \text{WER} = 0.25.$$

25% does not mean 25% of audio was bad — one edit per four reference words.

**Pause and predict.** Reference “WER and CER matter.” Hypothesis “WER matter.”  $\text{WER} = 2/4 = 0.5$ .

# Character Error Rate, Normalization, RTF

CER applies the same edit-distance idea to characters:

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c}.$$

- Useful when word boundaries are ambiguous, languages don't space the same way, or when a short technical token matters.

**Normalization rule:** lowercase, punctuation, whitespace, numerals, symbols. State the rule before reporting WER/CER, and use the same rule for all compared runs.

$$\text{RTF} = \frac{\text{processing time}}{\text{audio duration}}, \quad \text{RTF} < 1 \text{ is faster than real time.}$$



# Quality and Speed Together

- Lower WER but  $\text{RTF} = 3.0$  may be unsuitable for live use.
- Higher WER but  $\text{RTF} = 0.2$  may be ideal for batch indexing.
- Streaming: separate time-to-first-partial from finalization delay.

Accuracy alone is not enough. Report WER or CER beside latency, RTF, hardware, and memory. Applied deep learning is a tradeoff between quality, speed, memory, reproducibility, and the cost of the next experiment.

## End-to-End Objectives: CTC, Attention, Transducer

---

Old recipe: separate acoustic + pronunciation + language-model pieces.

Modern: one network mapping audio features to output tokens directly. Alignment doesn't disappear — it is handled inside the objective or decoder.

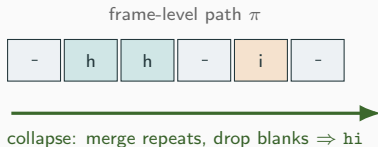
Three objective families:

**CTC** Sum over monotonic frame-level paths with blanks.

**Attention encoder-decoder** Generate tokens with cross-attention to encoder.

**Transducer (RNN-T-style)** Emit tokens while jointly tracking audio progress.

# CTC: Many Frame Paths, One Transcript

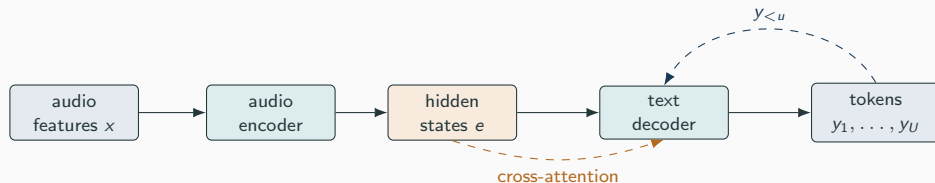


Other paths that collapse to hi: h\_\_i\_, hh\_ii\_, h\_iii\_, ...

$$p_{\text{CTC}}(y \mid x) = \sum_{\pi: B(\pi)=y} \prod_{t=1}^T p(\pi_t \mid x, t), \quad \mathcal{L}_{\text{CTC}} = -\log p_{\text{CTC}}(y \mid x).$$

CTC sums over frame paths — training never assigns one label per frame.

# Attention Encoder-Decoder ASR



$$p_{\theta}(y \mid x) = \prod_{u=1}^U p_{\theta}(y_u \mid y_{<u}, e), \quad \mathcal{L}_{\text{attn}} = - \sum_u \log p_{\theta}(y_u \mid y_{<u}, e).$$

Soft alignment lives inside the decoder; classic example: Listen, Attend, Spell.

# Transducer (RNN-T-Style) for Streaming

Lattice over audio time  $t$  and output position  $u$ :

$$p(k \mid t, u) = \text{softmax}_k(J(h_t, g_u)).$$

Sum over valid alignments that emit target tokens + blanks:

$$\mathcal{L}_{\text{RNNT}} = -\log \sum_{\pi \in \mathcal{A}(x, y)} \prod_{(t, u) \in \pi} p(\pi_{t, u} \mid t, u).$$

Designed to emit tokens *as audio frames arrive*.

For live captions, transducer-style is usually the natural starting point — but the final choice still depends on measured latency and accuracy.

# Transformer ASR Architectures

---

# Why Audio Length Hurts Attention

Full self-attention over  $T$  audio frames forms  $T^2$  pairwise scores per head.

- $T = 1000$ : 1,000,000 scores.
- Subsample by 4:  $T = 250$ , 62,500 scores —  $16\times$  smaller.

Speech encoders therefore use convolutional front-ends, striding, pooling, or chunking before deep attention layers.

**Downsample before attention.** Reducing audio time steps lowers attention cost quadratically.



# Encoder–Decoder for Speech, Conformer Locality

## Transformer ASR

- Audio encoder over feature frames
- Text decoder with causal mask
- Cross-attention to encoded audio
- Source modality is audio, not source-language tokens

## Conformer

- Self-attention + convolutional modules
- Captures global context *and* local acoustic patterns
- Useful: neighboring frames are strongly related

Pretraining helps too: wav2vec 2.0 (self-supervised), Whisper (large-scale weak supervision). Both reduce the need to train from scratch.

# Names Are Not Results

An architecture name is not enough evidence. A report should record:

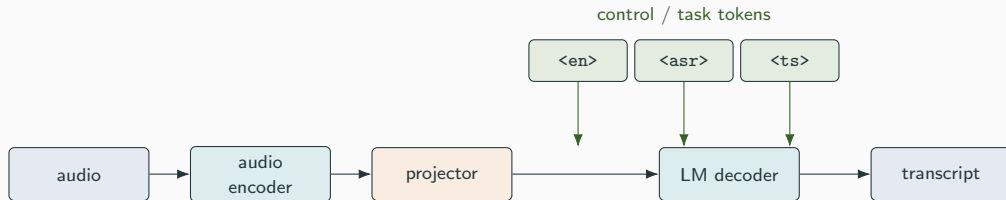
- Checkpoint, processor settings, precision, device.
- Batch size, max audio length, chunking rule.
- Whether run is offline or streaming.

Choose between checkpoints using local quality, speed, and error evidence on the target clips  
— not benchmark folklore.

# Audio-Language Models and Prompted ASR

---

# Audio-Language Models



Whisper-style conditioning: special tokens select language, task (transcribe vs translate), timestamps. *A glossary or prompt is not permission to invent words.*

## Context Biasing: A Useful Comparison

Run the same clips twice:

1. No domain prompt.
2. Short domain prompt.

Hold model, backend, precision, and normalization fixed.

Then compare:

- Domain-term errors (substitutions, deletions): expected to drop.
- Insertions during silence: may rise.

A good report names both sides of the tradeoff instead of quoting only the improved examples.

## Definition

Transcript text not supported by the audio. Common during silence, after music, around noise, or when a prompt strongly biases the decoder toward expected phrases.

Why language-model decoders make this worse:

- Traditional recognizers output rough fragments when uncertain.
- LM decoders produce fluent, plausible text that *sounds correct*.

**Fluency can mislead.** Plausible transcript text still needs audio support and reference-based scoring.

## Case Study: Qwen3-ASR

---

- Transformer-based ASR family from the Qwen team.
- Brings together: audio encoders, LM decoding, multilingual recognition, language identification, offline + streaming, forced alignment.
- Released checkpoints: Qwen3-ASR-0.6B, Qwen3-ASR-1.7B; plus Qwen3-ForcedAligner-0.6B (non-autoregressive).
- Reported support: ASR + language ID for 52 languages and dialects.

Treat package interfaces as implementation facts to verify. Model cards move faster than textbook concepts.



# Controlled Comparisons Suggested by Qwen3-ASR

**Size** 0.6B vs 1.7B: speed vs accuracy.

**Mode** Offline (more future context) vs streaming (low delay).

**Language** Auto language ID vs forced known language.

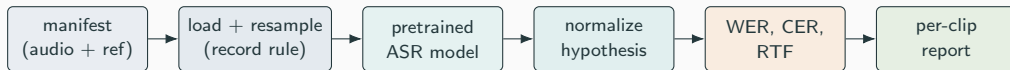
**Timestamps** Plain transcript vs transcript + forced alignment.

**Benchmarks are context, not conclusion.** Public scores motivate a baseline; local clips decide whether the system fits the task.

## Pipeline, Errors, and Recipe

---

# An ASR Run Is a Pipeline



Each arrow is a place where a hidden choice changes the result. The manifest, loader, decoder, normalizer, and scorer are all part of the experiment. **Save per-clip rows: averages hide which condition caused the result.**

**Acoustic** Echo, clipping, low volume, music, overlap.

**Domain vocabulary** Technical terms, names, acronyms (descent/decent).

**Language** Wrong language chosen, code-switch dropped, transliterated.

**Boundary / timestamp** Words right but timing drifts.

**Hallucination** Inserted words not in the audio (silence, noise, music).

**Decoding / prompt-induced** Insertions caused by context bias.

**Average WER hides cause.** Split errors by condition before choosing the next experiment.

# Debugging Map: Failure → Check → Next Run

| Observed failure             | First check                               | Next experiment                                    |
|------------------------------|-------------------------------------------|----------------------------------------------------|
| High WER on all clips        | Normalization rule, reference quality     | Re-score after fixing transcript or normalization  |
| High WER only on noisy clips | Listen for echo, clipping, music, overlap | Segment audio, improve recording, more robust ckpt |
| Wrong technical terms        | Error table for names, acronyms           | Add a short context prompt, compare model size     |
| Inserted prompt terms        | Audio support for each inserted term      | Shorten or remove prompt, inspect silence regions  |
| Slow offline transcription   | RTF, batch, precision, backend            | Smaller model, lower precision, faster backend     |
| Streaming lag                | Time-to-first-partial, finalization delay | Compare chunk size or streaming mode               |

## A Bounded Reference Run on 12 LibriSpeech Clips

Normalization: lowercase, ASCII letters/digits/apostrophes, collapsed whitespace. 8 validation, 4 reserved final-test.

| Run                     | WER    | CER    | RTF   | Dominant errors  |
|-------------------------|--------|--------|-------|------------------|
| Qwen3-ASR 0.6B          | 1.89%  | 0.47%  | 0.092 | acoustic + other |
| Qwen3-ASR 1.7B          | 1.31%  | 0.45%  | 0.116 | acoustic + other |
| 1.7B + domain prompt    | 4.01%  | 3.77%  | 0.073 | acoustic + other |
| 1.7B streaming API mode | 25.17% | 16.78% | 0.250 | acoustic + other |

Lesson: the table earns its claim by separating size, backend, prompting, streaming mode, speed, and error class.

# Practical Recipe

1. Name the use case; freeze a manifest with validation + final-test clips.
2. Write the normalization rule; run a pretrained baseline.
3. Change *one factor*: prompt, language setting, checkpoint, backend, precision, streaming mode.
4. For each run, record: ckpt, backend, device, precision, sample rate, language, prompt, decoding settings, software versions, WER, CER, total time, RTF; streaming adds partial-output latency + finalization delay.
5. Listen to one weak clip before reading the next model choice.
6. Select via validation, then run the final-test clips *once*.

# Summary

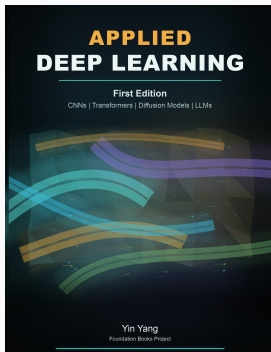
---



# Summary

- ASR maps sampled audio to text. The input is a waveform, not a sentence.
- Output contracts vary: text, language tags, timestamps, no-speech handling, offline vs streaming.
- Evaluation is WER + CER + normalization + speed (RTF, latency).
- End-to-end objectives: CTC sums over frame paths, attention enc-dec generates with cross-attention, transducers stream.
- Transformer / Conformer encoders adapt attention to long audio via subsampling and local structure; pretraining (wav2vec 2.0, Whisper) reduces from-scratch training.
- Audio-language models + prompted ASR can help domain vocabulary — and can also hallucinate.
- Qwen3-ASR is a modern case study, not a verdict.
- Discipline: define use case, manifest, baseline, normalize, score, measure speed, classify errors, change one factor, lock the final test.

**Next: Chapter 24 reverses direction — text-to-speech with transformer and codec language models.**



## Applied Deep Learning

Chapter overview slides for the book by Yin Yang.

|           |                                                                                                                                     |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------|
| Book page | <a href="https://foundation-books.github.io/applied-deep-learning">foundation-books.github.io/applied-deep-learning</a>             |
| Code      | <a href="https://github.com/foundation-books/applied-deep-learning-code">github.com/foundation-books/applied-deep-learning-code</a> |
| Print     | <a href="https://amazon.com/dp/B0GZF7QRSH">amazon.com/dp/B0GZF7QRSH</a>                                                             |
| Kindle    | <a href="https://amazon.com/dp/B0GZF9221X">amazon.com/dp/B0GZF9221X</a>                                                             |
| License   | CC BY-NC-ND 4.0                                                                                                                     |

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.